데이터베이스(SQL)

12장. DML

학습 목표

1) INSERT / UPDATE / DELETE의 역할을 설명할 수 있다.
2) RETURNING으로 변경된 행을 바로 확인할 수 있다.
3) UPDATE/DELETE 전에 SELECT로 대상을 확인하는 습관을 익힌다.
4) BEGIN / COMMIT / ROLLBACK으로 변경을 안전하게 테스트할 수 있다.
5) ON CONFLICT를 이용한 UPSERT의 목적을 이해한다.

전체 흐름

1) **DML**
챕터 01

→

2) **INSERT**
챕터 02

→

3) **UPDATE**
챕터 03

→

4) **DELETE**
챕터 04

5) **Transaction**
챕터 05

→

6) **UPSERT**
챕터 06

01 DML

DDL과 DML

구분	DDL	DML
바꾸는 것	구조 (테이블의 틀)	행 데이터 (틀 안의 값)
명령	CREATE, ALTER, TRUNCATE, DROP	INSERT, UPDATE, DELETE
예	컬럼을 하나 추가한다	주문 금액을 1,000원 올린다

공식 문서: 데이터 조작

안전 수칙 5단계



1) 먼저 SELECT로 대상 확인

2) 필요하면 BEGIN

3) UPDATE / DELETE

4) RETURNING 또는 SELECT로 결과 확인

5) 맞으면 COMMIT, 연습이면 ROLLBACK

02 INSERT

INSERT 기본 형식

한 행

```
INSERT INTO 테이블명 (컬럼1, 컬럼2)
VALUES (값1, 값2);
```

여러 행

```
INSERT INTO 테이블명 (컬럼1, 컬럼2)
VALUES (값1, 값2),
(값3, 값4);
```

- 컬럼 목록과 VALUES의 값은 **순서와 개수가 맞아야** 한다.
- 여러 행은 괄호 묶음을 쉼표로 이어 한 번에 넣는다.
- 문자와 날짜 값은 작은따옴표로 감싼다.

공식 문서: INSERT

INSERT 옵션

형식	뜻
INSERT INTO 테이블명 VALUES (…);	컬럼 목록을 생략하면 지정한 값이 테이블에 정의된 컬럼 순서대로 대응된다. 일부 컬럼의 값만 지정할 경우 앞쪽 컬럼부터 대응하며, 생략된 컬럼에는 기본값이 적용된다.
컬럼 목록에서 빠진 컬럼	선언한 기본값(DEFAULT), 없으면 NULL이 들어간다
VALUES (값1, DEFAULT)	그 자리에 기본값을 명시적으로 넣는다
INSERT INTO 테이블명 (컬럼1, 컬럼2) SELECT 컬럼1, 컬럼2 FROM 다른테이블 WHERE 조건;	조회 결과를 행으로 넣는다 (0행 이상)
… RETURNING *;	방금 넣은 행을 결과로 바로 돌려받는다

컬럼 목록은 생략할 수 있지만, 테이블 구조가 바뀌면 순서가 어긋나므로 **컬럼 목록을 적는 습관**을 들인다.

03 UPDATE

UPDATE 기본 형식

```
UPDATE 테이블명
SET 컬럼1 = 값1,
컬럼2 = 값2
WHERE 조건;
```

- SET에 적은 컬럼만 바뀌고, 나머지 컬럼은 그대로다.
- WHERE 조건을 만족하는 **모든 행**이 바뀐다.

기존 값을 바탕으로 계산

```
SET amount = amount + 1000
```

공식 문서: UPDATE

WHERE 없는 UPDATE의 위험

WHERE를 빠뜨리면

```
UPDATE 테이블명
SET 컬럼1 = 값1;
```

테이블의 **모든 행**이 같은 값으로 바뀐다.

바꾸기 전에 같은 조건으로 확인

```
SELECT *
FROM 테이블명
WHERE 조건;
```

RETURNING으로 즉시 확인

```
UPDATE 테이블명
SET amount = amount + 1000
WHERE 조건
RETURNING *;
```

- RETURNING은 실제로 바뀐 행을 **바뀐 뒤의 값**으로 돌려준다.

- RETURNING * 대신 RETURNING 컬럼1, 컬럼2처럼 필요한 컬럼만 고를 수 있다.

- 추가 SELECT 없이 결과를 확인할 수 있어, 트랜잭션 안에서 COMMIT 여부를 판단할 때 편하다.

04 DELETE

DELETE 기본 형식

1) 삭제 전 SELECT

```
SELECT *
FROM 테이블명
WHERE 조건;
```

2) 같은 조건으로 DELETE + RETURNING

```
DELETE FROM 테이블명
WHERE 조건
RETURNING *;
```

- WHERE 조건을 만족하는 모든 행이 지워진다.
- RETURNING *은 **지워진 행**을 돌려준다. 무엇이 지워졌는지 바로 확인한다.
- 실행 결과로 지운 행 수가 나온다. 0행이어도 오류가 아니다.

공식 문서: DELETE

WHERE 누락 시 전체 행 삭제

```
DELETE FROM 테이블명;
```

WHERE가 없으면 **테이블의 모든 행**이 지워진다. 테이블은 남지만 비어 있다.

	DELETE (WHERE 없음)	TRUNCATE
결과	모든 행 삭제, 구조 유지	모든 행 삭제, 구조 유지
조건	WHERE로 일부만 지울 수 있다	조건 없이 전체만
속도	행을 하나씩 지운다	전체를 지울 때 더 빠르다

05 Transaction

BEGIN, COMMIT, ROLLBACK

```
BEGIN;

UPDATE 테이블명
SET 컬럼1 = 값1
WHERE 조건
RETURNING *;

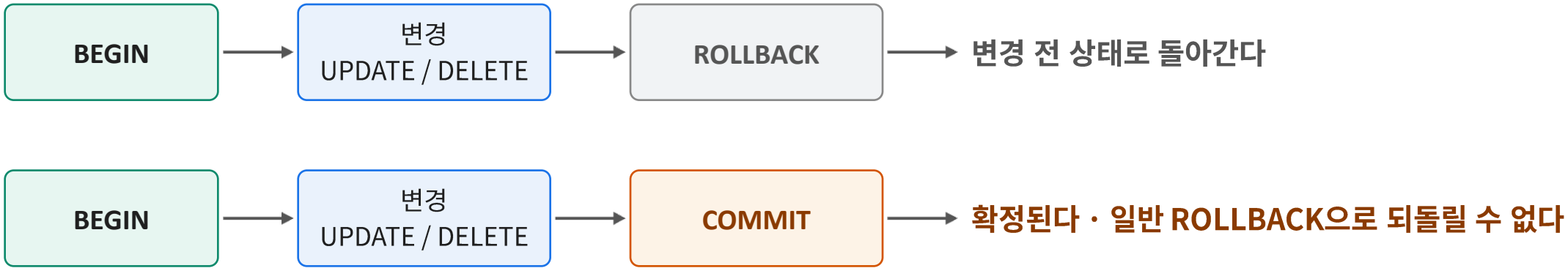
ROLLBACK; -- 또는 COMMIT;
```

명령	하는 일
BEGIN	트랜잭션을 시작한다
COMMIT	그때까지의 변경을 확정한다
ROLLBACK	그때까지의 변경을 모두 취소한다

COMMIT 전에는 다른 사용자에게 변경이 보이지 않고, ROLLBACK으로 되돌릴 수 있다.

공식 문서: 트랜잭션

COMMIT 전에는 되돌릴 수 있다



COMMIT 전에는 되돌릴 수 있다

Autocommit 주의

DBeaver 설정에 따라 문장이 **실행되자마자 자동으로 확정**될 수 있다.

DBeaver 모드	동작	되돌리기
Auto-commit	문장마다 실행 직후 바로 확정한다	ROLLBACK할 수 없다
Manual commit	COMMIT 또는 ROLLBACK을 할 때까지 변경이 열린 트랜잭션에 남는다	ROLLBACK할 수 있다

공식 문서: DBeaver Transaction mode

06 UPSERT

INSERT … ON CONFLICT

PostgreSQL의 INSERT … ON CONFLICT는 UNIQUE/PK 충돌 시 DO NOTHING 또는 DO UPDATE를 선택할 수 있다.

충돌하면 고치기 (UPSERT)

```
INSERT INTO 테이블명 (키컬럼, 컬럼2)
VALUES (값1, 값2)
ON CONFLICT (키컬럼)
DO UPDATE SET 컬럼2 = EXCLUDED.컬럼2;
```

충돌하면 건너뛰기

```
INSERT INTO 테이블명 (키컬럼, 컬럼2)
VALUES (값1, 값2)
ON CONFLICT (키컬럼) DO NOTHING;
```

부분	뜻
ON CONFLICT (키컬럼)	이 컬럼 값이 이미 있으면 충돌로 본다. 그 컬럼에 PK나 UNIQUE가 있어야 한다
DO UPDATE SET …	기존 행을 고친다. 이때는 괄호 안 충돌 기준을 반드시 적는다
EXCLUDED.컬럼	넣으려다 충돌한 새 행의 값

공식 문서: INSERT

제조 예제 - 설비 상태 최신값

설비 상태 테이블 (설비코드 PK)

설비코드	상태
M001	정상

현재 저장된 최신값

INSERT →

설비코드	상태
M001	**점검**

다시 입력한 값 (EXCLUDED)

M001 충돌 →

설비코드	상태
M001	**점검**

DO UPDATE 후: 점검으로 갱신

ON CONFLICT (설비코드) DO UPDATE SET 상태 = EXCLUDED.상태

RETURNING 한눈에

명령	형식	돌려주는 행
INSERT	INSERT INTO … VALUES (…) RETURNING *;	실제로 넣은 행
UPDATE	UPDATE … SET … WHERE … RETURNING *;	실제로 바뀐 행 (바뀐 뒤 값)
DELETE	DELETE FROM … WHERE … RETURNING *;	실제로 지워진 행
INSERT … ON CONFLICT DO UPDATE	… DO UPDATE SET … RETURNING *;	넣거나 고친 행

INSERT / UPDATE / DELETE 결과를 **추가 SELECT 없이** 확인한다. PostgreSQL 분석과 운영에서 유용하다.

공식 문서: 변경된 행의 데이터 반환